FirstSlip: finding the first wrong step in student algebra with a symbolic checker

Oct 15, 2026 · @Abdul Rehman

Summary

FirstSlip measures whether LLM tutors can find the first wrong step in a student's algebra solution, and trains a small open model to do it better. A SymPy checker labels every step of synthetic solutions exactly. These labels are used in three ways: to score off-the-shelf LLMs, to ground the feedback they write, and as the reward for training a small model.

The trained model is then tested on 1,002 student solutions whose first error step was annotated by teachers, where no checker applies. A small web tutor shows learners which parts of its feedback the checker has verified and which are the model's own guess.

The plan runs for about five and a half weeks at 15 hours per week (roughly 80 hours), in seven phases numbered 0 to 6. A complete, usable result exists at the end of Phase 3; Phases 4 and 5 add the training and transfer results.

Problem and gap

LLMs solve grade-school maths well but often cannot say where a student went wrong. On the VtG set of 1,002 teacher-annotated solutions, GPT-4o located the first wrong step in 54.5% of cases without a reference solution, against 18.3% for random guessing (Srivatsa et al., 2025). Tyen et al. (2024) report the same pattern: models correct an error well once told where it is, but rarely find it themselves.

This matters for tutoring because feedback aimed at the wrong step may send a learner to fix something that was correct. In addition, students who practised with an unguarded GPT-4 tutor performed worse once it was removed (Bastani et al., 2025), so a tutor that sounds confident but points at the wrong step is a real risk.

What prior work already covers

Work	What it shows	What it leaves open
Daheim et al., EMNLP 2024	Grounding feedback in a separate verifier makes tutor responses more targeted; releases VtG	Labels are human annotations; no symbolic checker; no training from a verifiable reward
Srivatsa et al., 2025	A corrected copy of the student's solution helps models locate the first error	Prompting only; no training
ProcessBench, 2024	Step-level error identification on competition maths	Model-written solutions, not student errors
MalAlgoPy, 2024	Generates linear-equation solutions with 20 documented student misconceptions	Trains models to imitate students, not to find their errors; code not public
MalAlgoLib, 2026	Models learn where an error enters only when trained on intermediate steps	Studies misconception acquisition, not error localization or feedback

To our knowledge, no published work uses an exact symbolic checker both as the label source and as the training reward for locating student errors, and then tests whether the trained model transfers to teacher-annotated errors where no checker applies. We also found no test of whether an LLM tutor repeats a verifier's location when that location is wrong. FirstSlip addresses these three points.

One caveat shapes the design. The VtG solutions were written by an LLM prompted to act as a student (MathDial), and teachers then marked the first error. They are word problems rather than symbolic algebra, so they serve as a realistic transfer test, not as real student work.

Research questions and hypotheses

The project asks four questions. Each hypothesis is written down before any run, as in the Tagalog study, so that a null result is reported as a null result.

RQ1. Measurement. How often do current LLMs locate the first wrong step in a student's algebra solution, name the misconception, and write feedback about the same step they flagged?

- H1: Localization accuracy on synthetic algebra will be well below the models' accuracy at solving the same equations, as reported for word problems.
- H1b: In a measurable share of responses, the written feedback discusses a step other than the one the model flagged.

RQ2. Grounding and trust. Does giving the model the checker's error location improve its feedback, and when that location is deliberately wrong, how often does the model repeat it?

- H2: Correct locations raise feedback accuracy, consistent with Daheim et al. (2024).
- H2b: Models will adopt a wrong location in most cases rather than contradict it.

RQ3. Training from the checker. Does reinforcement learning with the checker as reward improve error localization in a small open model, compared with supervised fine-tuning (SFT) on the same labels?

- H3: Both improve over the base model on misconception types seen in training. Only a model that learned to check steps, rather than to recognise error patterns, will hold its gain on held-out misconception types.
- Anti-hypothesis: the gain reduces to a position prior (guessing the most common error step for a given solution length). A position-only baseline tests this directly.

RQ4. Transfer and side effects. Does the gain carry over to the teacher-annotated VtG set, where no checker applies? Does rewarding only the step number change how faithful the model's explanations are?

- H4: Transfer to VtG will be positive but much smaller than the in-domain gain, because VtG errors are mostly misreadings of word problems rather than algebraic slips.
- H4b: No prediction is made about faithfulness; the direction of the change is the finding.

Fit with prospective supervisors

Each part of FirstSlip answers a question one supervisor already works on, so the same project can be described to each without changing its substance. Dr. Ye's page describes her work as how language models reason and how to make that reasoning more reliable,

through post-training, inference techniques and evaluation benchmarks (homepage). Dr. Demmans Epp's group builds adaptive learning technologies, learning analytics, and ways of communicating information about learning.

Project part	Dr. Xi Ye	Dr. Carrie Demmans Epp
Symbolic step checker	Verifiable signal for reasoning; solver does the checking, LLM does the explaining (SatLM)	Reliable diagnosis of student errors
Localization benchmark	Evaluation benchmark for a reasoning skill models lack	Diagnosing where a learner went wrong
Faithfulness measure	Whether explanations match the model's own judgement (Ye and Durrett, 2022)	Whether feedback is trustworthy for learners
Corrupted-tool control	Tool trust in tool-augmented reasoning	Over-reliance on AI tutors (Bastani et al., 2025)
GRPO with checker reward	Post-training from a verifiable reward; robustness on held-out types	Misconception-aware models (MalAlgoLib)
Transfer to VtG	Reasoning beyond the verifiable domain it was trained in	Feedback on messier, more realistic student work
Student app	Inference-time use of a verifier	Showing learners which claims are verified and which are guesses (open learner models)

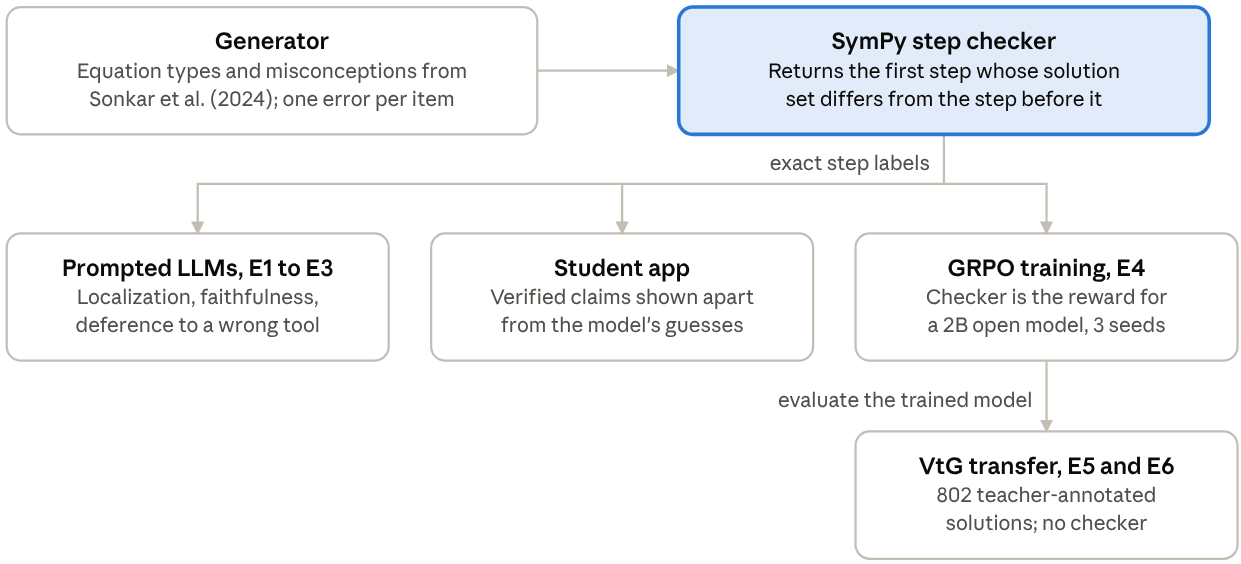
For Dr. Ye, lead with the reasoning question: a benchmark where every error location is known by construction, and a small model trained on that signal. For Dr. Demmans Epp, lead with the tutor: feedback that separates what it knows from what it guesses, and a learner study that the project makes possible but does not run.

Components and tools

Every component runs on free or open tools, and the only running cost is API calls for the evaluated models and the app. One finding changes the plan: MalAlgoPy's code is not public (a third-party survey reports that every candidate GitHub repository returns 404),

so the generator is written from scratch from the misconception table in Sonkar et al. (2024). This takes about two days and leaves the generator fully under your control.

One checker labels every step, and three parts of the project use its labels



how FirstSlip fits together · one checker, three uses

The checker is the only source of labels; the trained model is then tested on VtG, where the checker cannot run.

Component	Tool	Licence or cost	Why this one
Solution and error generator	Own Python module implementing MalAlgoPy's 16 equation types and its misconception table	Your code	Code not released; the published table of misconceptions and the reduction graph are enough to rebuild it
Step checker	SymPy: `solveset` for equations, `simplify(a - b) == 0` plus random numeric spot checks for expressions	BSD	Exact, deterministic, already in Python

Component	Tool	Licence or cost	Why this one
Input parsing	`sympy.parsing.sympy_parser.parse_expr` for plain text; `parse_latex(..., backend="lark")` for app input	BSD	The lark backend avoids the ANTLR version pin
Teacher-annotated test set	VtG on Hugging Face (1,002 rows; first error index, error category, teacher description)	CC BY 4.0	The standard first-error benchmark for student solutions
Optional harder test	PRM800K test split	MIT	Only if time remains; competition-level maths
Evaluated API models	One frontier model and one low-cost model, versions pinned and logged	Pay per call; set a spending cap and cache every response	Shows whether model scale changes the pattern
Open models	Qwen3.5-2B and Qwen3.5-4B, served with vLLM	Apache 2.0	Released March 2026; small enough for a free T4
RL and SFT training	TRL `GRPOTrainer` with custom reward functions; `SFTTrainer` for the baseline	Apache 2.0	The checker plugs in directly as a reward function
Memory savings	Unsloth with LoRA	Apache 2.0	Fits a 2B model and GRPO sampling in 16 GB
Compute	Kaggle notebooks (2 x T4, roughly 30 GPU hours a week; the quota floats); Colab free tier as backup	Free	Enough for three seeds of a 2B model

Component	Tool	Licence or cost	Why this one
Logging	JSONL per run with git commit, seed and config hash	Free	Same discipline as the Tagalog study
Statistics	Paired bootstrap (10,000 resamples), McNemar test, Holm correction	Free	Methods you already used
App backend	FastAPI	MIT	You built a FastAPI service for ScholarAssistant
App maths input	MathLive web component	MIT	Students type maths naturally; it outputs LaTeX
Hosting	Hugging Face Spaces (free CPU) for the app; GitHub and a Hugging Face dataset card for the release	Free	No server to manage

If Qwen3.5's Gated DeltaNet layers do not run efficiently on a T4, Qwen3-1.7B is the fallback; the plan does not depend on a specific model family.

Data

All training data is synthetic and labelled by the checker; the only human-labelled data is VtG, which is used for testing alone. Sizes below are targets for the generator, not measurements.

Split	Source	Planned size	Use
Train	Generator, 15 of the misconceptions	6,000 incorrect + 2,000 clean	SFT and GRPO
Dev	Generator, same 15 misconceptions	500	Prompt tuning, early stopping, the GRPO go/no-go check

Split	Source	Planned size	Use
Test, seen types	Generator, same 15 misconceptions, new equations	1,000 (25% clean)	In-domain results
Test, held-out types	Generator, the remaining misconceptions only	1,000 (25% clean)	Did the model learn to check, or to recognise patterns?
VtG dev	VtG, random 200 rows	200	Prompt format only; never used for training
VtG test	VtG, remaining rows	802	Transfer test

Rules the generator must satisfy

1. Each incorrect solution contains exactly one injected misconception, and the checker must confirm that the solution set changes at that step.
2. No later step may return to the correct answer by chance; items where it does are dropped and counted.
3. The first-error position is balanced across step indices, so that a position-only guess cannot score well.
4. Equations are deduplicated by their canonical SymPy form across all splits.
5. Coefficients are integers between -12 and 12, answers are integers or simple fractions, and solutions run three to six steps.

Held-out misconceptions

Hold out one misconception from each family so the held-out test is not trivially easy or hard: one distribution error (for example, distributing to the first term only), one sign error, one division error (for example, dividing one side only), and one term-combining error. Fix this choice before generating any data, and record it in the repository.

VtG handling

Report results on all 802 test rows and, separately, without the "None of the above" category (8.7% of VtG), whose labels are the least consistent. Because VtG problems come from GSM8K, evaluated models may have seen the questions in pre-training; this

affects solving more than locating a specific simulated student's error, but it is stated as a limitation.

Experiments and metrics

Six experiments answer the four research questions; E1 to E3 need no training and E4 to E6 do.

Exp.	RQ	Conditions	Data
E1 Prompted localization	RQ1	Random; position-only; 2 API models; 2 open models	Test seen, test held-out, VtG test
E2 Grounded feedback	RQ2	No location given vs. checker location given; LLM writes the feedback	Synthetic tests
E3 Corrupted tool	RQ2	Location given is the true step, one step off, or a random wrong step	Synthetic tests
E4 Training	RQ3	Base vs. SFT vs. GRPO (3 seeds each); rejection-sampling fine-tuning if GRPO fails its gate	Train, then both synthetic tests
E5 Transfer	RQ4	Same checkpoints as E4	VtG test
E6 Faithfulness after training	RQ4	Same checkpoints, asked to explain the flagged step	Synthetic tests and VtG test

Output format

Every model answers in one fixed format: the first wrong step number (or 0 for no error), then one sentence of feedback. A fixed format keeps parsing exact and keeps GRPO completions short enough for a T4.

Reward for GRPO

The checker scores each completion. A correct step number earns 1.0; a correct "no error" on a clean solution earns 1.0; a step number off by one earns 0.2; anything else earns 0. A separate format reward of 0.1 applies when the answer parses. Nothing in the reward looks at the feedback sentence, which is what makes E6 a clean test.

Metrics

- **First-error accuracy:** exact match with the true first wrong step, on incorrect solutions.
- **Clean accuracy:** saying "no error" on correct solutions.
- **Error F1:** the harmonic mean of the two accuracies above, following ProcessBench, so a model cannot score well by always flagging an error.
- **Within-one accuracy:** for comparison with Srivatsa et al. (2025).
- **Misconception accuracy:** naming the injected misconception (synthetic data only), scored by a fixed mapping from free text to labels, audited by hand on 100 items.
- **Faithfulness:** the share of responses whose feedback sentence refers to the step they flagged. A rule-based extractor finds the step mentioned; 100 items are checked by hand, and the extractor's agreement with you is reported.
- **Deference rate (E3):** the share of responses that adopt the supplied wrong location.

Statistics

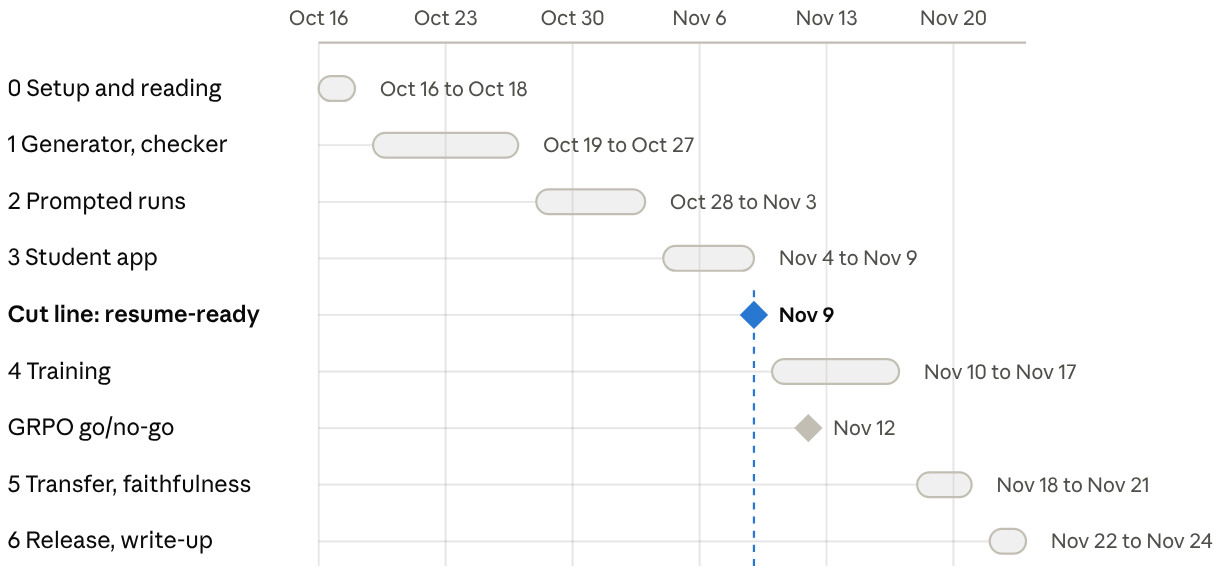
Model comparisons on the same items use a paired bootstrap with 10,000 resamples and McNemar's test, with Holm correction within each experiment. For trained models, report the mean and standard deviation over three seeds, and treat a difference as stable only when it exceeds the between-seed standard deviation, the same criterion as the Tagalog paper.

Phased plan

The work runs in seven phases from 16 October to 24 November 2026, and each phase ends with a gate you must pass before starting the next. Phases 0 to 3 produce a complete project on their own (benchmark, prompted results and a live app) by 9 November; Phases 4 to 6 add training and transfer. If the application deadline falls earlier, stop at the cut line and list the project as ongoing.

A complete project exists by 9 November; training and transfer follow

FirstSlip phases by date, 16 October to 24 November 2026



phase timeline · 16 Oct to 24 Nov 2026

Phases 0 to 3, which end at the cut line, stand on their own; Phases 4 to 6 add the training and transfer results.

Phase 0. Setup and core reading (16 to 18 Oct, about 6 hours)

- [ ] Create the GitHub repository, a Python environment with pinned versions, and a Kaggle notebook with GPU enabled
- [ ] Load VtG from Hugging Face and inspect 30 rows by hand
- [ ] Compute the random and position-only baselines on VtG
- [ ] Read items 1 to 5 of the reading list
- [ ] Write the hypotheses and the held-out misconception choice into `PREREGISTRATION.md` and commit it

Gate: both VtG baselines computed and the pre-registration file committed.

Phase 1. Generator and checker (19 to 27 Oct, about 16 hours)

- [ ] Implement the 16 equation types as reduction rules, each producing one step
- [ ] Implement the misconceptions from Sonkar et al. (2024), Table 1, as alternative steps
- [ ] Write the checker: parse each line, compare solution sets between consecutive steps, return the first step where they differ

- [ ] Add the generator rules (one error per item, no accidental recovery, balanced positions, deduplication)
- [ ] Generate all synthetic splits and write a dataset card

Gate: on 2,000 generated items, the checker finds the injected step in 100% of incorrect items and reports no error on 100% of clean items; 50 items checked by hand.

Phase 2. Prompted experiments E1 to E3 (28 Oct to 3 Nov, about 15 hours)

- [ ] Write one prompt template and the output parser; freeze both on the dev splits
- [ ] Run E1 on four models across both synthetic tests and VtG test, caching every response
- [ ] Run E2 and E3 on the synthetic tests
- [ ] Build the faithfulness extractor and audit 100 items by hand
- [ ] Produce result tables with bootstrap intervals

Gate: all E1 to E3 tables filled, and the extractor agrees with your hand labels on at least 90% of the audited items.

Phase 3. Student app (4 to 9 Nov, about 12 hours)

- [ ] FastAPI endpoint: problem plus steps in, per-step check and feedback out
- [ ] Single page with MathLive input, one line per step, green or red marks per step
- [ ] Feedback panel that labels checker-verified claims separately from the model's guesses
- [ ] Deploy to Hugging Face Spaces with a per-session request cap
- [ ] A results page with the E1 to E3 findings

Gate: a friend can solve, make a mistake and get feedback on a phone. This is the cut line: the resume entry can be written from here.

Phase 4. Training E4 (10 to 17 Nov, about 16 hours)

- [ ] SFT baseline on the train split (one run per seed)
- [ ] GRPO with the checker reward, starting from the base model
- [ ] **Go/no-go on 12 Nov:** if mean reward has not risen and dev accuracy is not above the base model, switch to rejection-sampling fine-tuning (sample several answers, keep those the checker confirms, fine-tune on them)
- [ ] Three seeds for each trained condition

- [ ] Evaluate on both synthetic tests; plot the distribution of predicted steps to check for position bias

Gate: three seeds per condition evaluated, with held-out results and the position check reported.

Phase 5. Transfer and faithfulness E5 and E6 (18 to 21 Nov, about 8 hours)

- [ ] Evaluate all checkpoints on the VtG test, overall and by error category
- [ ] Run the faithfulness measure on trained models
- [ ] Write a one-paragraph answer to each research question, including the nulls

Gate: every hypothesis marked supported, not supported or inconclusive, with its evidence.

Phase 6. Release and write-up (22 to 24 Nov, about 9 hours)

- [ ] Release code, generator and synthetic data (GitHub and a Hugging Face dataset card)
- [ ] A short technical report (four to six pages) using the humanizer-academic style
- [ ] A 60-second screen recording of the app
- [ ] Update the resume entry and the statement-of-purpose paragraph

Gate: a stranger can reproduce the E1 table from the repository README.

Risks and fallbacks

The largest risk is GRPO giving no learning signal on a small model; the go/no-go check on 12 November limits the cost to two days.

Risk	What could happen	Fallback
GRPO has no signal	Every sampled answer in a group gets the same reward, so the update is zero	Start with easier items (two-step equations), raise the number of samples per prompt, or switch to rejection-sampling fine-tuning
Model does not run well on a T4	Qwen3.5's newer layers are slow or unsupported on older GPUs	Use Qwen3-1.7B; the design does not depend on the model family

Risk	What could happen	Fallback
Generator takes longer than planned	Rebuilding the misconception rules is fiddly	Start with the 8 most common misconceptions and add the rest later; the held-out design still works
Position shortcut	A trained model guesses a common step number rather than checking	Balanced positions in the data, the position-only baseline, and a plot of predicted positions
Noisy VtG labels	Some teacher annotations are inconsistent	Report results with and without the "None of the above" category
Parsing failures in the app	Students type steps the parser cannot read	MathLive input, one equation per line, and a clear "could not read this line" message rather than a guess
API cost	Many models across thousands of items	Cache every response, cap spending, and run the full grid only on the final prompt
Kaggle quota	Weekly GPU hours run out during Phase 4	Keep each GRPO run under 3 hours; use Colab free as overflow
Overclaiming	A synthetic result is presented as a learning outcome	State plainly that no learner study was run, and that VtG solutions are simulated

No learner data is collected. The app keeps no logs by default, so no ethics review is needed; a learner study is listed as future work for the MSc.

Deliverables and resume entry

By the cut line on 9 November you will have the first four deliverables below; the last three come from Phases 4 to 6.

1. An open-source generator and SymPy checker for algebra solutions with known first errors

2. A benchmark of synthetic test sets (seen and held-out misconceptions) with a dataset card
3. Prompted results for four models on localization, faithfulness and deference to a wrong verifier
4. A live student app that separates verified feedback from the model's guesses
5. Trained checkpoints (SFT and GRPO, three seeds each) with held-out and VtG results
6. A four- to six-page technical report
7. A 60-second screen recording of the app

Resume entry

Fill in the numbers after Phase 5. Use the short version if you stop at the cut line.

> **FirstSlip: verifiable error localization for LLM tutors** (live app, GitHub). Built a SymPy step checker and a controlled benchmark of N algebra solutions with known first errors across K misconception types. Showed that prompted LLMs locate the first error in X% of cases and adopt a deliberately wrong verifier location in Y% of cases. Trained a 2B model with GRPO using the checker as reward: A% to B% on held-out misconception types and C% on 802 teacher-annotated solutions (SFT: D%).

Short version, at the cut line: replace the last sentence with "Deployed a tutor that marks which feedback the checker has verified."

One sentence for each statement of purpose

- **Dr. Ye:** I built a benchmark in which the location of every reasoning error is known by construction, and used it to test whether a verifiable step-level reward trains a small model to find errors that transfer to human-annotated data.
- **Dr. Demmans Epp:** I built a tutor that tells learners which parts of its feedback a symbolic checker has confirmed, and found that LLM tutors repeat a wrong verifier's diagnosis in Y% of cases; a learner study of how students interpret this distinction is what I would like to run in EdTeKLA.

Reading list

Read the first group in order during Phase 0 (about 8 hours); the other groups are matched to the phase that needs them. Tick each item off as you finish it.

A. Must read first, in this order (Phase 0)

- [ ] **Stepwise Verification and Remediation of Student Reasoning Errors with LLM Tutors** (Daheim et al., EMNLP 2024). Read all of it. The closest prior work and the source of VtG; your project must say clearly what it adds.
- [ ] **LLMs cannot spot math errors, even when allowed to peek into the solution** (Srivatsa, Maurya and Kochmar, 2025). Read Sections 1 to 4 and the prompt appendix. Gives the VtG baselines you compare against and prompt templates you can reuse.
- [ ] **LLMs cannot find reasoning errors, but can correct them given the error location** (Tyen et al., Findings of ACL 2024). Read the abstract, Sections 1 and 4. The finding your tool-augmented condition builds on.
- [ ] **LLM-based Cognitive Models of Students with Misconceptions** (Sonkar et al., 2024). Read Section 3 and Table 1 closely. This is the blueprint for your generator.
- [ ] **Misconception Acquisition Dynamics in Large Language Models** (Liu et al., 2026). Read the abstract and results. Shows that intermediate steps are what let a model learn where an error enters.

B. Dr. Ye's work (Phase 0 and before writing the statement of purpose)

- [ ] **The Unreliability of Explanations in Few-shot Prompting for Textual Reasoning** (Ye and Durrett, NeurIPS 2022). The origin of your faithfulness measure.
- [ ] **SatLM: Satisfiability-Aided Language Models Using Declarative Prompting** (Ye, Chen, Dillig and Durrett, NeurIPS 2023). The pattern of LLM plus solver that your grounded condition follows.
- [ ] **To CoT or not to CoT? Chain-of-thought helps mainly on math and symbolic reasoning** (Sprague et al., ICLR 2025). You have read it; reread the section comparing CoT with symbolic solvers.

C. Step-level evaluation and verifiable rewards (Phases 2 and 4)

- [ ] **ProcessBench: Identifying Process Errors in Mathematical Reasoning** (Zheng et al., 2024). Read Sections 1 to 3. Source of the error F1 metric.
- [ ] **Let's Verify Step by Step** (Lightman et al., 2023). Read Sections 1 to 3. Process supervision and the PRM800K dataset.
- [ ] **DeepSeekMath** (Shao et al., 2024). Read the GRPO section only. The algorithm you will train with.
- [ ] **TRL GRPOTrainer documentation**. Read the custom reward function section before writing any training code.

D. The learner side (Phases 3 and 6)

- [ ] **Generative AI without guardrails can harm learning** (Bastani et al., PNAS 2025). The evidence behind your corrupted-tool control.
- [ ] **On Assessing the Faithfulness of LLM-generated Feedback on Student Assignments** (Jia et al., EDM 2024). Prior work on feedback faithfulness from the education side.
- [ ] **Uncertainty Representation in Visualizations of Learning Analytics for Learners** (Demmans Epp and Bull, IEEE TLT 2015). Why the app labels verified and guessed claims.
- [ ] **Representing Uncertainty in Open Learner Models: How Well Do People Interpret It?** (Demmans Epp et al., AIED 2026). The study your future learner experiment would extend.
- [ ] **Algebra Mal-rules and Cognitive Accounts of Error** (Payne and Squibb, Cognitive Science 1990). The original source of the misconception taxonomy.
- [ ] **MathDial** (Macina et al., Findings of EMNLP 2023). Read Section 3 to understand how the VtG student solutions were produced.

Sources

Pages opened while preparing this proposal, as of 15 October 2026:

- VtG dataset card
- Daheim et al., EMNLP 2024
- Srivatsa et al., 2025
- Sonkar et al., 2024 (MalAlgoPy)
- Liu et al., 2026 (MalAlgoLib)
- Note that MalAlgoPy has no public repository
- ProcessBench
- TRL GRPOTrainer documentation
- Qwen3.5 small models
- Qwen3.5 small models, licence
- Unsloth notebooks
- Kaggle GPU quota
- SymPy LaTeX parser
- Xi Ye homepage